Linear: The Anti-Feature Company

Teaching Case (HBS-style) — written for this course

Prepared for: TUM Entrepreneurship Course: Prototyping Entrepreneurial Ideas for AI

Decision Point: Summer 2020 — 500 Paying Teams, 100 Feature Requests, What to Build Next?

How this case ties into Module 10

This module asks what you refuse to build; Linear is the company that made refusal a strategy. Work three connections as you read: (1) **Idea 1 — the 3-Question Filter:** run it live against the Gantt-chart request in the case — the same filter you'll run on your own backlog in the Feature Graveyard workshop. (2) **Idea 2 — the North Star:** Linear's discipline only works because they know what their one metric is; name it from the case evidence. (3) **Idea 4 — default alive changes the question:** with 33+ months of runway, Linear never had to say yes for survival — check Exhibit A, then ask what YOUR runway does to your feature decisions this week.

Introduction

Most successful product companies follow a predictable trajectory: launch with core features, listen to customers, add the features they request, scale.

Karri Saarinen, a designer and engineer who spent years at Airbnb watching product development from both inside and outside, believed this trajectory was a trap.

In 2019, Saarinen left Airbnb and founded Linear, a project management tool, together with two fellow Finns: Tuomas Artman (ex-Uber) and Jori Lallo (ex-Coinbase). But his philosophy was fundamentally different: **Linear would succeed by saying NO to features more often than saying YES.**

By summer 2020, Linear had 500 paying teams and was growing rapidly. Users loved it. But the feature requests were piling up:

- **Gantt charts** (every team demands this eventually)

- **Time tracking** (needed for consultancies and agencies)
- **Resource management** (who's available for what?)
- **Custom fields** (every team has unique data)
- **Mobile app** (everyone's on the go)
- **Advanced reporting** (C-suite needs dashboards)
- **Integration with X** (accounting, HR, CRM, etc.)

Jira, the entrenched competitor, had all of these features — and hundreds more.

Saarinen faced a decision that would define Linear's entire future: **In a world of infinite possible features, how do you choose what to build?**

Background

The Problem: Project Management Is Bloated (2018-2019)

By 2018, project management software had become a graveyard of feature creep.

Market leaders and their approach:

Product	Founded	Philosophy	Problem
Jira	2002	"Enterprise completeness"	Too many features. 5 clicks to create a task. Overkill for most teams.
Monday.com	2012	"Flexible customization"	Endless options paralyze users. Slow UI. Consumes CPU.
Asana	2008	"Work OS for teams"	Tries to be everything (tasks, projects, portfolio, resource management). Does nothing perfectly.
Trello	2011	"Simple kanban"	So simple it's useless for complex workflows. Acquired by Atlassian, now bloating.
Notion	2016	"All-in-one workspace"	Can do anything, but requires you to build everything. Steep learning curve.

The pattern: most tools started simple, then added features for every edge case and customer request, until they became bloated and slow.

A developer writing in a blog post in 2018 summed it up:

> "Jira has become a second job. I spend 20 minutes a day just updating statuses and filling out fields. The tool is supposed to help us communicate, but instead it's become a chore we resent."

The insight: Teams didn't want more features. They wanted the right features, implemented well, without the cognitive overhead of the wrong features.

Karri Saarinen's Vision

Karri Saarinen had a specific background that shaped his thinking:

Airbnb (until 2019) - Started as designer, evolved into founding member of Airbnb's design systems team - Observed how Airbnb (a company famous for taste and intentionality) made product decisions - Learned that intentionality scales: saying no to small things allows you to say yes to big things - Witnessed how "feature creep" killed products from the inside

Influenced by: - Paul Graham's essays on design and simplicity - Dieter Rams' principle: "Less but better" - The philosophy of Japanese craftsmanship: master one thing before moving to the next

The thesis: Most software companies optimize for breadth (more features, more customers, more revenue). But the winners optimize for depth (fewer features, mastered completely).

Saarinen's bet: In 2020, teams were so exhausted by bloated tools that a **deliberately minimal project management tool** would be perceived as luxury.

Building Linear (2019-2020)

Saarinen, Artman, and Lallo founded Linear in 2019 and built it on these core principles:

The Linear Principles: 1. **Intentional scope.** Only build features that solve the core problem: collaboration and task tracking. 2. **Speed above all else.** The UI should feel instant. No spinners, no lag. 3. **Keyboard-first.** Power users should never need a mouse. 4. **Beautiful by default.** Design is not optional; it's fundamental. 5. **No customization without depth.** Don't offer custom fields unless they're essential. Don't add options to appease edge cases.

What Linear Did NOT Build (2019-2020): - No Gantt charts - No time tracking - No resource management - No portfolio/roadmap management - No "custom fields" (not initially) - No mobile app - No advanced reporting

What Linear DID Build: - Fast, beautiful task and issue tracking - Teams and projects - Cycles (simple sprints) - Issue relationships (blocking, related, duplicates) - Automation and workflows - A powerful search - API for integrations - Slack integration

The Result: - Shipped in less time than competitors spent on planning - Felt 10x faster than Jira - Users discovered Linear through word-of-mouth raving about the speed - By mid-2020: 500 paying teams, 15% month-over-month growth

The Competitor Context

By 2020, the market looked like this:

Category	Leader	Weakness
Enterprise (Jira)	Atlassian	Bloated, slow, expensive
SMB (Monday.com)	Monday.com	Too flexible, confusing
Startups (Trello)	Atlassian (acquired)	Too simple
All-in-one (Notion)	Notion	Does everything poorly
Premium minimalist	Nobody	**Gap in market**

Linear was the first premium project management tool that offered less, not more.

The Situation

The Requests Piling Up

By summer 2020, Linear's Slack channel and feature request board were filling with asks:

From customers:

> "Linear is amazing, but we need Gantt charts for our product roadmap. We're evaluating Jira just because of this."
>
> "How do we track how much time each task took? We need that for invoicing clients."
>
> "Can we add a custom field for 'Client' and 'Project Code'? Every task needs both."

> "When's the mobile app coming? My team's in the field and needs to update issues on their phone."
>
> "We need something like Monday's automations or Zapier. Integrating everything manually is killing us."
>
> "Any chance of an API for custom reporting? We need to send data to our accounting system."

From investors and board: Linear had raised a seed round ($4.2M in late 2019, led by Sequoia) and was in conversations for a Series A. VCs asked pointed questions:

> "Your competitor Jira has 500+ features. How do you compete with breadth?" "What's your retention curve? Feature-heavy tools have lower churn because customers invest in customization." "Have you thought about verticalization? Maybe you become the project tool for designers or agencies specifically?"

From within Linear's team: Saarinen's small engineering team (8 people) was productive: Linear shipped constantly — small improvements and fixes every week — but had capacity for only a couple of major feature bets per year (each roughly 6 months of work). But which ones?

The paradox: Linear was growing fast partly because it had fewer features. But growth also meant pressure to add more features to retain customers and compete with entrenched players.

The Data: What Are Customers Actually Using?

To make intentional decisions, Saarinen and team analyzed actual usage data.

Feature usage (% of active teams using the feature in a month):

Illustrative reconstruction for classroom use — directionally faithful, individual numbers are teaching estimates.

Feature	Usage	Notes
Create/edit issues	98%	Core
Assign issues	87%	Core
Change status	85%	Core
Add comments	72%	High
Link issues (block/relate)	41%	Moderate
Custom views/filters	38%	Moderate
Cycles (sprints)	31%	Niche
Automation	12%	Very niche
API usage	3%	Very small

Key insight: Most teams use Linear for core task tracking. Advanced features are used by small subsets. Yet the feature requests come from those subsets, creating pressure to build for the minority.

The Feature Request Breakdown

Saarinen tracked all feature requests for the past 6 months:

Illustrative reconstruction for classroom use — directionally faithful, individual numbers are teaching estimates.

Request	# of Requests	% of Customer Base Asking	Assessment
Gantt charts	47	9%	Important for specific verticals (product, ops) but optional for most
Time tracking	31	6%	Important for agencies/ consultancies, but adds complexity
Custom fields	28	5%	Each team wants different fields — will explode scope
Mobile app	25	5%	Everyone wants it eventually, but is it needed for task management?
Better reporting	19	4%	Mostly for C-level visibility, not daily users
Zapier/advanced integrations	17	3%	Power users with complex workflows
Portfolio/roadmap view	16	3%	For product managers and executives
Advanced automation	14	3%	Complex workflows in large orgs
Resource management	12	2%	Critical for agencies, irrelevant for product teams

Key insight: No single feature request represents more than 10% of the customer base. But collectively, there's a long tail of requests that could easily consume all engineering bandwidth.

The Strategic Question: Vertical Focus vs. Horizontal Broadening

Linear's team debated two strategies:

Strategy A: Vertical Focus Linear could become the project management tool for a specific vertical: - **For product teams:** Add roadmap views, better reporting, portfolio management - **For software engineers:** Add advanced CI/CD integration, more sophisticated automation - **For agencies:** Add time tracking, project billing, Gantt charts, resource management

Pros: Laser focus, build features that integrate deeply, win a specific market Cons: Abandon the horizontal (multi-vertical) market, potentially smaller TAM, competitors can go vertical too

Strategy B: Horizontal Broadening Linear could say yes to the most popular feature requests (Gantt, time tracking, mobile) and become a more complete project management platform.

Pros: Larger TAM, appeal to more customer segments, fight Jira's breadth Cons: The whole reason Linear succeeded was not being broad. Broadening might destroy what makes it special.

Strategy C: Maintain Course Saarinen believed there was a third way: build features that serve the majority of users better, not edge cases.

The questions: - What features do 50%+ of Linear users actually need? - What features are we not building that would make the core experience better? - How do we grow into adjacent features without losing our soul?

The Three-Question Filter

To make decisions intentionally, Saarinen had been developing a decision framework — later documented in writing and talks. The framework:

Question 1: Does this solve a core problem that blocks many users?

Gantt charts: 9% of users ask for it. But 91% don't mention it. Do they not need it, or do they just expect project management tools to not have it?

Question 2: Can we build it beautifully and fast, or does it require compromise?

Time tracking: Accurate time tracking requires integrations with calendar, automations, API calls to measure activity. This is complex. It risks slowing down the whole product for 6% of users.

Question 3: Does this align with our identity, or does it dilute us?

Mobile app: Everyone wants a mobile app. But building a mobile app is hard (iOS and Android engineering). And Linear's value (speed, keyboard shortcuts) doesn't translate to mobile. Building a mediocre mobile app might hurt Linear's brand.

The Paradox of Intentionality

Saarinen faced a paradox that applies to all product companies:

The “Vibe-Coding” Death Spiral:

1. You start with strong principles (Linear = fast, minimal, beautiful)
2. You get customers who like your principles
3. Customers say “I like you because you’re fast, but I also need X”
4. You add X, compromising on speed/minimalism slightly
5. New customers come who like X, but don’t care about speed
6. Eventually you’re optimizing for the wrong customers
7. You become a slower, bloated version of Jira

The only way to avoid this spiral: **Make choices and accept that some customers will leave.**

The Decision Point

It is **summer 2020**. Linear has 500 paying teams. Growth is strong. Saarinen is preparing for Series A fundraising and needs to show a clear product vision.

He sits down to decide: **What does Linear build in the next 12 months?**

The engineering team can realistically ship: - 2 major features (6 months work each) - 4-6 medium features (4-6 weeks work each)

Or some combination.

The Choices Before Him

Option A: Build Gantt Charts (Breadth Expansion) - Addresses the most common feature request (9% of users ask for it, which seems small, but represents significant revenue risk) - Directly competitive with Jira/Asana - 6-8 weeks of engineering work - Requires UI changes (adding timeline view to Linear’s fast, minimal interface) - Might make Linear feel less minimal

Pros: - Unlock product teams (product managers are high-value, sticky customers) - Show investors Linear can serve multiple verticals - Answer the most frequent ask

Cons: - Doesn’t improve the core experience for the 91% who don’t use Gantt - Risk visual and performance regression in core product - What’s next? Investors will want time tracking, resource management, reporting...

Option B: Double Down on Speed and Quality (Depth Focus) - Instead of new features, improve performance, API, automation, integrations - Build better search, better filtering,

better issue relationships - Build webhooks and integrations that let customers extend Linear for their custom needs - Polish the 31% of users using Cycles to make them amazing

Pros: - Strengthen what makes Linear special (speed, simplicity) - Build a platform that's extensible without getting bloated - Win by being better at core tasks, not wider

Cons: - Doesn't address feature requests directly - Investors might see this as lack of ambition - Competitors might capture the vertical-specific niches

Option C: Verticalize Around One Segment (Strategic Focus) - Choose one vertical (e.g., software engineering teams) and build depth - For engineers: advanced CI/CD integration, deployment tracking, code-related automation - Stop being "project management for everyone," become "issue tracking for engineers" - Leverage network effects within the engineering community

Pros: - Clear positioning and differentiation - Sustainable competitive advantage (hard for generalist tools to compete) - Easier to understand and market

Cons: - Abandon the horizontal TAM (might be a $10B+ opportunity) - Alienate product teams, design teams, marketing teams (current customers) - Pick wrong vertical, and you're stuck

Option D: Verticalize Around Agencies (Growth Play) - Agencies need: time tracking, project billing, resource management, client portals - This is a distinct need from software engineering or product management - Agencies are willing to pay premium prices - It's an underserved market

Pros: - High-revenue vertical (agencies bill by the hour) - Distinct customer needs, not a diluted market - Less competition (Jira doesn't care about agencies, Asana is trying but not succeeding)

Cons: - Time tracking is complex and risky to build - Changes the product for everyone (billing, invoicing, client management) - Might cannibalize the software engineering use case (core early users)

Option E: Build the Mobile App (Market Expansion) - Everyone's on the go - Investors see mobile as table stakes - 5% of users asking for it will turn into 50% if it exists

Pros: - Unlock new use cases (capturing tasks on the go, notifications) - Show investors Linear is thinking about modern work (not stuck on desktop) - Mobile-first teams might adopt if they have good mobile

Cons: - Massive engineering effort (iOS + Android is 3-4 months minimum) - Linear's value (speed, keyboard shortcuts) doesn't translate to mobile - Building mediocre mobile is worse than not building at all

What Would Change Saarinen's Mind?

If certain data existed, it would change the decision:

- **If 30%+ of users asked for Gantt charts:** Saarinen would build it
- **If usage analysis showed core feature stagnation:** Saarinen would focus on depth
- **If engineering sentiment showed burnout:** Saarinen would slow down
- **If investors said "we'll only fund if you're in 3+ verticals":** Saarinen might reconsider direction
- **If a specific vertical (e.g., design teams) showed 80% conversion rate:** Saarinen might verticalize

The Real Question

Behind the feature choice is a deeper strategic question Saarinen must answer:

What is Linear's identity?

- A horizontal project management platform competing with Jira?
- A premium tool for software engineers (serving a critical niche)?
- A beautifully-designed tool for teams that value simplicity?
- A developer-centric tool (with APIs and webhooks, letting customers build custom)?
- A domain-specific platform for a vertical (agencies, product teams, etc.)?

The answer to this question determines which features get built.

If Linear is "a horizontal project management platform," then Gantt, time tracking, mobile become necessary.

If Linear is "the beautifully-designed tool for teams who value simplicity," then these features might be the wrong answer.

Exhibits

Exhibit A: Linear's Core Metrics (Summer 2020)

Illustrative reconstruction for classroom use — directionally faithful, individual numbers are teaching estimates.

Metric	Value	Benchmark
MRR	~$40-50k	(estimate)
ARR	~$500-600k	(estimate)
Paying teams	500	—
Avg revenue per team	$1k-1.2k/year	Well above Asana ($400/yr), below enterprise Jira
Monthly growth rate	15%	Exceptional for B2B SaaS
Net retention rate	120%	Strong (churn is low, expansion revenue is high)
Team size	8 engineers, 3 others	Very lean
Cash in bank	~$4M (seed round)	~33-36 months of runway at current burn — default-alive
Burn rate	~$100-120k/month	(estimated)

Key insight: Run the math: ~$4M in the bank at ~$100-120k/month gross burn is roughly **33-36 months of runway** — closer to three years than to three months, and longer still once MRR offsets burn. Linear was default-alive. It did not need to say yes to revenue-driven feature demands to survive. That removes the comfortable excuse ("we had no choice, we needed the revenue") and sharpens the question this case actually turns on: **what do you refuse to build, and what does that refusal cost you?**

Exhibit B: Customer Segmentation

Illustrative reconstruction for classroom use — directionally faithful, individual numbers are teaching estimates.

Segment	% of Customer Base	Key Needs	Churn Risk	Expansion Potential
Software teams (engineers)	35%	Fast task tracking, CI/CD integration, automation	Low	High (tools like Linear become central)
Product teams	25%	Roadmap visibility, cycle planning, reporting	Medium	Medium (need Gantt, but use elsewhere)
Design/ marketing teams	15%	Simple task tracking, collaboration	Medium	Low (Linear is nice but not core)
Agencies/ services	12%	Time tracking, project billing, client access	High	High (if Linear adds agency features)
Early-stage startups	13%	Low-cost, beautiful tool	High	Medium (scale to competitors)

Exhibit C: Feature Request Sentiment Analysis

Illustrative reconstruction for classroom use — directionally faithful, individual numbers are teaching estimates.

Analysis of qualitative feedback from feature requests:

Theme	Sentiment	Example
Speed/ performance	Positive	“Linear is snappy, I love it”
Simplicity	Positive	“Finally a tool that doesn’t try to do everything”
Design	Positive	“The UI makes me happy”
Missing Gantt	Negative	“Love Linear but it’s a non-starter without Gantt”
Missing time tracking	Negative	“We tried Linear but had to switch because we bill by the hour”
Mobile	Neutral/Negative	“Need mobile for when we’re not at desk”
Missing custom fields	Negative	“Every team has different needs; can’t fit everyone”
API limitations	Negative (from power users)	“Wish I could build custom automations via API”
Comparison to Jira	Mixed	“Why Linear instead of Jira? Because it’s not slow/bloated”

Exhibit D: Engineering Capacity Scenarios

Illustrative reconstruction for classroom use — directionally faithful, individual numbers are teaching estimates.

Scenario A: Continue Current Pace (weekly small improvements, ~2 major feature bets per year) - Q3-Q4 2020: Build feature 1 - Q1-Q2 2021: Build feature 2 - Allows deep focus, clean implementation, low bug risk

Scenario B: Accelerated (3 major features per year) - Hire 3-4 more engineers (take burn rate to $150-180k/month) - Can tackle Gantt + time tracking + mobile app in 12 months - Risk: quality regression, burnout, diluted focus

Scenario C: Focused Single Vertical (1 major feature + 3 medium) - Q3-Q4 2020: Build time tracking and project billing (agency focus) - Q1-Q2 2021: Build 3 agency-specific features - Full pivot to agencies — abandon horizontal market

Scenario D: Platform First (0 new features, 12 months of API/automation) - Build webhooks, GraphQL API, advanced automation - Let customers build custom features on top - Accept that we can't be everything; let ecosystem extend us

Exhibit E: Investor Expectations (Series A)

Based on typical investor asks in 2020:

Investor Type	Expectation	Risk Level
Venture (a16z, Spark Capital)	"Horizontal SaaS play. Grow to enterprise. $100M+ ARR."	High (asks for breadth)
Mid-market focused (Sapphire, Thrive)	"Vertical play. Dominate agencies or engineering. $50M ARR."	Medium (focused strategy)
Founder-focused (Sequoia, Benchmark)	"Build what you believe. Don't compromise vision for growth."	Low (strategy agnostic)

The investor you take capital from influences the strategy you'll be pressured to execute.

Exhibit F: Competitive Landscape — Feature Breadth vs. Depth

Product	# Features	Speed Rating	Ease of Use	Premium Focus
Jira	500+	Slow	Hard	Enterprise, complexity
Monday.com	200+	Medium	Medium	SMB flexibility
Asana	150+	Medium	Medium	Teams, portfolios
Notion	~100 (flexible)	Medium	Hard	All-in-one
Trello	30	Fast	Easy	Simplicity (degraded)
Linear	25	Very Fast	Very Easy	Premium minimalism

Key insight: Linear has succeeded by being the opposite of Jira. If Linear adds 100 features, it becomes a worse version of Jira, not a better Jira.

Questions for Class Discussion

1. **The data vs. the principle:** Only 9% of Linear users ask for Gantt charts. But as Linear grows, this number will increase. At what threshold (20%? 30%?) do you build a feature customers ask for, even if it violates your core principles?
2. **The verticalization question:** Is it better to be a horizontal tool that's great at one core workflow, or a vertical tool that's great for one customer segment? What are the long-term implications of each choice?
3. **The identity risk:** Linear's identity is speed and simplicity. But growth pressure (hiring, investors, customers) will pull toward breadth. How does a company protect its identity as it scales?
4. **The investor influence:** If Series A investors say "build a mobile app" or "add Gantt charts," how much influence should they have? Is the founder betting against investor judgment a good move?
5. **The "best tool" vs. "enough tool" question:** Is Linear better because it does fewer things, or is it good-enough at enough things that customers are willing to switch? What happens when someone builds a tool that's 95% as fast as Linear but has Gantt and time tracking?
6. **The churn/expansion question:** Linear's net retention is 120%, which means customers expand (pay more, more teams). This suggests Linear can add features without losing customers. Does this mean the team should add more features, or that it can afford to stay minimal?
7. **The "lack of features as marketing" problem:** Linear's minimalism is part of why it's attractive. But once Linear is no longer novel (everyone's heard of it), will minimalism be a disadvantage? Is the positioning sustainable long-term?

Epilogue (read after class)

Spoiler warning: this section gives away what Linear actually did. Read it after the class discussion, not before.

For historical context, Linear's actual strategy was a mix of Options B and D:

Saarinen continued to focus on core depth (faster search, better filtering, webhooks) while selectively adding features. The team grew slowly and intentionally. By 2022-2023, Linear:

- Did not add free-form custom fields — one of the loudest requests — shipping opinionated primitives instead (labels, priorities, estimates, project views) that cover most needs without exploding scope
- Built a GraphQL API allowing customers to build integrations and automations
- Developed webhooks and automation workflows
- Did not build a traditional mobile app for years, focusing on the desktop and web experience where its speed advantage lives (a deliberately lightweight mobile app came much later)
- Did not build Gantt charts initially (later added, but as a secondary view)
- Did not verticalize, but instead became the tool for software engineering teams

The company's $13M Series A (December 2020) was led by Sequoia — which had also led the $4.2M seed — and backed the founders' vision over breadth pressure. This allowed Linear to stay disciplined.

The later funding history rewards a close look precisely because it is so undramatic: a $35M Series B led by Accel in September 2023 at a valuation of roughly $400 million, then an $82M Series C in 2025 at a $1.25 billion valuation — about $134M raised in total, by a company known for being profitable/default-alive with a deliberately small team. Linear never needed the money to survive; it raised on its own terms. By the mid-2020s, Linear had become the quality bar for developer tools — "Linear-quality" is shorthand for craft in B2B software — and it is still famous for what it refuses to build.

The case illustrates that sometimes the most strategic decision is not to build something, and that this requires tremendous discipline and clarity of identity.

Teaching case written for this course from public sources. Exhibits marked as illustrative are reconstructions for classroom use.